JavaScript is disabled on your browser.

compact1, compact2, compact3

java.lang

Class Process

java.lang.Object
    java.lang.Process

```
public abstract class Process
extends Object
```

The `ProcessBuilder.start()` and `Runtime.exec` methods create a native process and return an instance of a subclass of `Process` that can be used to control the process and obtain information about it. The class `Process` provides methods for performing input from the process, performing output to the process, waiting for the process to complete, checking the exit status of the process, and destroying (killing) the process.

The methods that create processes may not work well for special processes on certain native platforms, such as native windowing processes, daemon processes, Win16/DOS processes on Microsoft Windows, or shell scripts.

By default, the created subprocess does not have its own terminal or console. All its standard I/O (i.e. stdin, stdout, stderr) operations will be redirected to the parent process, where they can be accessed via the streams obtained using the methods `getOutputStream()`, `getInputStream()`, and `getErrorStream()`. The parent process uses these streams to feed input to and get output from the subprocess. Because some native platforms only provide limited buffer size for standard input and output streams, failure to promptly write the input stream or read the output stream of the subprocess may cause the subprocess to block, or even deadlock.

Where desired, subprocess I/O can also be redirected using methods of the `ProcessBuilder` class.

The subprocess is not killed when there are no more references to the `Process` object, but rather the subprocess continues executing asynchronously.

There is no requirement that a process represented by a `Process` object execute asynchronously or concurrently with respect to the Java process that owns the `Process` object.

As of 1.5, `ProcessBuilder.start()` is the preferred way to create a `Process`.

Since:

`JDK1.0`

Constructor Summary

Constructors

Constructor	Description
`Process()`	

Method Summary

Abstract Methods

Concrete Methods

Modifier and Type	Method	Description
`abstract void`	`destroy()`	Kills the subprocess.
`Process`	`destroyForcibly()`	Kills the subprocess.
`abstract int`	`exitValue()`	Returns the exit value for the subprocess.
`abstract InputStream`	`getErrorStream()`	Returns the input stream connected to the error output of the subprocess.
`abstract InputStream`	`getInputStream()`	Returns the input stream connected to the normal output of the subprocess.
`abstract OutputStream`	`getOutputStream()`	Returns the output stream connected to the normal input of the subprocess.
`boolean`	`isAlive()`	Tests whether the subprocess

		represented by this `Process` is alive.
abstract int	**waitFor**()	Causes the current thread to wait, if necessary, until the process represented by this `Process` object has terminated.
boolean	**waitFor**(long timeout, **TimeUnit** unit)	Causes the current thread to wait, if necessary, until the subprocess represented by this `Process` object has terminated, or the specified waiting time elapses.

Methods inherited from class java.lang.Object

clone, equals, finalize, getClass, hashCode, notify, notifyAll, toString, wait, wait, wait

Constructor Detail

Process

```
public Process()
```

Method Detail

getOutputStream

```
public abstract OutputStream getOutputStream()
```

Returns the output stream connected to the normal input of the subprocess. Output to the stream is piped into the standard input of the process represented by this `Process` object.

If the standard input of the subprocess has been redirected using `ProcessBuilder.redirectInput` then this method will return a null output stream.

Implementation note: It is a good idea for the returned output stream to be buffered.

Returns:

`the output stream connected to the normal input of the subprocess`

getInputStream

```
public abstract InputStream getInputStream()
```

Returns the input stream connected to the normal output of the subprocess. The stream obtains data piped from the standard output of the process represented by this `Process` object.

If the standard output of the subprocess has been redirected using `ProcessBuilder.redirectOutput` then this method will return a null input stream.

Otherwise, if the standard error of the subprocess has been redirected using `ProcessBuilder.redirectErrorStream` then the input stream returned by this method will receive the merged standard output and the standard error of the subprocess.

Implementation note: It is a good idea for the returned input stream to be buffered.

Returns:

`the input stream connected to the normal output of the subprocess`

getErrorStream

```
public abstract InputStream getErrorStream()
```

Returns the input stream connected to the error output of the subprocess. The stream obtains data piped from the error output of the process represented by this `Process` object.

If the standard error of the subprocess has been redirected using `ProcessBuilder.redirectError` or `ProcessBuilder.redirectErrorStream` then this method will return a null input stream.

Implementation note: It is a good idea for the returned input stream to be buffered.

Returns:

the input stream connected to the error output of the subprocess

waitFor

```
public abstract int waitFor()
                    throws InterruptedException
```

Causes the current thread to wait, if necessary, until the process represented by this `Process` object has terminated. This method returns immediately if the subprocess has already terminated. If the subprocess has not yet terminated, the calling thread will be blocked until the subprocess exits.

Returns:

the exit value of the subprocess represented by this Process object. By convention, the value 0 indicates normal termination.

Throws:

`InterruptedException` - if the current thread is `interrupted` by another thread while it is waiting, then the wait is ended and an `InterruptedException` is thrown.

waitFor

```
public boolean waitFor(long timeout,
                       TimeUnit unit)
                throws InterruptedException
```

Causes the current thread to wait, if necessary, until the subprocess represented by this `Process` object has terminated, or the specified waiting time elapses.

If the subprocess has already terminated then this method returns immediately with the value `true`. If the process has not terminated and the timeout value is less than, or equal to, zero, then this method returns immediately with the value `false`.

The default implementation of this methods polls the `exitValue` to check if the process has terminated. Concrete implementations of this class are strongly encouraged to override this method with a more efficient implementation.

Parameters:

`timeout` - the maximum time to wait

`unit` - the time unit of the timeout argument

Returns:

true if the subprocess has exited and false if the waiting time elapsed before the subprocess has exited.

Throws:

`InterruptedException` - if the current thread is interrupted while waiting.

`NullPointerException` - if unit is null

Since:

1.8

exitValue

```
public abstract int exitValue()
```

Returns the exit value for the subprocess.

Returns:

the exit value of the subprocess represented by this Process object. By convention, the value 0 indicates normal termination.

Throws:

`IllegalThreadStateException` - if the subprocess represented by this Process object has not yet terminated

destroy

```
public abstract void destroy()
```

Kills the subprocess. Whether the subprocess represented by this `Process` object is forcibly terminated or not is implementation dependent.

destroyForcibly

```
public Process destroyForcibly()
```

Kills the subprocess. The subprocess represented by this `Process` object is forcibly terminated.

The default implementation of this method invokes `destroy()` and so may not forcibly terminate the process. Concrete implementations of this class are strongly encouraged to override this method with a compliant implementation. Invoking this method on `Process` objects returned by `ProcessBuilder.start()` and `Runtime.exec(java.lang.String)` will forcibly terminate the process.

Note: The subprocess may not terminate immediately. i.e. `isAlive()` may return true for a brief period after `destroyForcibly()` is called. This method may be chained to `waitFor()` if needed.

Returns:

the Process object representing the subprocess to be forcibly destroyed.

Since:

1.8

isAlive

```
public boolean isAlive()
```

Tests whether the subprocess represented by this `Process` is alive.

Returns:

true if the subprocess represented by this Process object has not yet terminated.

Since:

1.8

Submit a bug or feature
For further API reference and developer documentation, see Java SE Documentation. That documentation contains more detailed, developer-targeted descriptions, with conceptual overviews, definitions of terms, workarounds, and working code examples.
Copyright © 1993, 2026, Oracle and/or its affiliates. All rights reserved. Use is subject to license terms.
Also see the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.